Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number: 2520030008
Name: Sagi Sahasra Srihasa

Generative AI
25CS2207E

LAB RECORD

2026

WEEK 3 – MONGODB CONNECTIVITY

AIM

To develop an LLM Monitoring System using Python, MongoDB, and Ollama that tracks and stores Large Language Model (LLM) usage details such as input tokens, output tokens, total tokens, response latency, cost, and request status in a MongoDB database.

OBJECTIVES

1. To establish connectivity between Python and MongoDB using PyMongo.
2. To integrate Large Language Models using a common provider interface.
3. To implement Gemini and Ollama provider classes.
4. To generate AI responses using the locally running Ollama model.
5. To calculate token usage and estimated API costs.
6. To measure response latency.
7. To store LLM request details in MongoDB.
8. To implement retry handling for temporary failures.

SOFTWARE REQUIREMENTS

- Python
- MongoDB Community Server
- MongoDB Compass
- Ollama
- Llama 3.2 Model
- Visual Studio Code
- PyMongo
- Requests
- Google GenAI
- Python-dotenv

DATABASE CONFIGURATION

Database Name: llm_monitoring

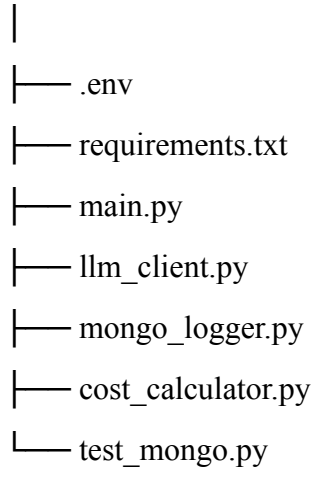
Collection Name: call_logs

MongoDB Connection String:

mongodb://localhost:27017/

PROJECT STRUCTURE

Week3/



├── .env

├── requirements.txt

├── main.py

├── llm_client.py

├── mongo_logger.py

├── cost_calculator.py

└── test_mongo.py

PROCEDURE

1. Download and install MongoDB Community Server and MongoDB Compass.
2. Start the MongoDB service and verify its status using PowerShell.
3. Open MongoDB Compass and connect to the local MongoDB server.
4. Create a database named `llm_monitoring` and a collection named `call_logs`.
5. Install the required Python libraries using the requirements file.
6. Install Ollama and download the Llama 3.2 model.
7. Create Python modules for cost calculation, LLM provider integration, MongoDB logging, and application execution.
8. Establish a connection between Python and MongoDB using PyMongo.
9. Execute the application and select Ollama as the LLM provider.
10. Enter a prompt and generate an AI response.
11. Store the prompt, token usage, response latency, cost, and execution status in MongoDB.
12. Verify the stored records using MongoDB Compass.

SOURCE CODE

1. cost_calculator.py

```
# cost_calculator.py


# Example rates per 1 million tokens
PRICING = {
    "gemini-3-flash-pro": {
        "input": 0.50,
        "output": 2.00
    }
}



def calculate_cost(provider, model, input_tokens, output_tokens):

    # Ollama runs locally and does not charge per token
    if provider == "OllamaProvider":
        return 0.0

    rates = PRICING.get(model)

    if rates is None:
        return None

    input_cost = (
        input_tokens * rates["input"]
    ) / 1_000_000
```

```
    output_cost = (
        output_tokens * rates["output"]
    ) / 1_000_000

    total_cost = input_cost + output_cost

    return round(total_cost, 10)
```

2. mongo_logger.py

```
# mongo_logger.py

from pymongo import MongoClient
from datetime import datetime, timezone


class MongoLogger:

    def __init__(self):

        self.client = MongoClient(
            "mongodb://localhost:27017/",
            serverSelectionTimeoutMS=5000
        )

        # Verify MongoDB connection
        self.client.admin.command("ping")

        self.db = self.client["llm_monitoring"]
```

```
        self.collection = self.db["call_logs"]

        print("Connected to MongoDB successfully!")

    def log_call(self, log_data):

        log_data["timestamp"] = datetime.now(timezone.utc)

        self.collection.insert_one(log_data)

        print("Call details saved in MongoDB!")

    def close(self):

        self.client.close()
```

3. llm_client.py

```
# llm_client.py

from abc import ABC, abstractmethod
from google import genai
from google.genai import types
import requests
import time

from cost_calculator import calculate_cost


# 1. Common LLM Provider Interface
```

```
class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass


# 2. Gemini Provider

class GeminiProvider(LLMProvider):

    def __init__(self, api_key):

        self.client = genai.Client(
            api_key=api_key,
            http_options=types.HttpOptions(timeout=30000)
        )

        self.model = "gemini-3-flash-pro"

    def generate(self, prompt):

        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt
        )
```

```
        usage = response.usage_metadata

        input_tokens = (
            usage.prompt_token_count if usage else 0
        )

        output_tokens = (
            usage.candidates_token_count if usage else 0
        )

        return {
            "response": response.text,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens
        }


# 3. Ollama Provider

class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):

        self.model = model

        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
```

```
        response = requests.post(
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=30
        )

        response.raise_for_status()

        data = response.json()

        return {
            "response": data["response"],
            "input_tokens": data.get("prompt_eval_count", 0),
            "output_tokens": data.get("eval_count", 0)
        }


# 4. Common LLM Client

class LLMClient:

    def __init__(self, provider, logger):
```

```
    self.provider = provider

    self.logger = logger

def generate(self, prompt):

    max_attempts = 3

    start_time = time.perf_counter()

    last_error = None

    for attempt in range(1, max_attempts + 1):

        try:

            # Generate AI response
            result = self.provider.generate(prompt)

            # Calculate elapsed time
            latency = round(
                (time.perf_counter() - start_time) * 1000,
                2
            )

            input_tokens = result["input_tokens"]

            output_tokens = result["output_tokens"]
```

```
model = self.provider.model

provider_name = type(self.provider).__name__

cost = calculate_cost(
    provider_name,
    model,
    input_tokens,
    output_tokens
)

# Prepare MongoDB document
log_data = {
    "provider": provider_name,
    "model": model,
    "prompt": prompt,
    "input_tokens": input_tokens,
    "output_tokens": output_tokens,
    "total_tokens": input_tokens + output_tokens,
    "latency_ms": latency,
    "cost": cost,
    "attempts": attempt,
    "status": "success",
    "error": None
}

# Save call details in MongoDB
```

```
        try:
            self.logger.log_call(log_data)
        except Exception as log_error:
            print("MongoDB logging error:", log_error)

        return result["response"]

    except Exception as error:

        last_error = error

        # Identify temporary failures
        status_code = getattr(error, "status_code", None)

        retryable = (
            isinstance(
                error,
                (
                    requests.exceptions.Timeout,
                    requests.exceptions.ConnectionError
                )
            )
            or status_code == 429
            or (
                isinstance(status_code, int)
                and status_code >= 500
            )
            or (
```

```
            isinstance(error, requests.exceptions.HTTPError)
            and error.response is not None
            and (
                error.response.status_code == 429
                or error.response.status_code >= 500
            )
        )
    )

    if not retryable or attempt == max_attempts:
        break

    wait_time = 2 ** (attempt - 1)

    print(
        f"Attempt {attempt} failed. "
        f"Retrying in {wait_time} seconds..."
    )

    time.sleep(wait_time)

# All attempts failed
latency = round(
    (time.perf_counter() - start_time) * 1000,
    2
)

failed_log = {
```

```
            "provider": type(self.provider).__name__,
            "model": self.provider.model,
            "prompt": prompt,
            "latency_ms": latency,
            "attempts": attempt,
            "status": "failed",
            "error": str(last_error)
        }

        try:
            self.logger.log_call(failed_log)
        except Exception as log_error:
            print("MongoDB logging error:", log_error)

        raise last_error
```

4. main.py

```
# main.py

import os

from dotenv import load_dotenv

from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)
```

```
from mongo_logger import MongoLogger


# Load environment variables
load_dotenv(override=True)


print("==================================")
print("       WEEK 3: LLM MONITORING")
print("==================================")

print("1. Gemini")
print("2. Ollama")

choice = input("\nSelect provider (1/2): ").strip()

logger = None

try:

    # Connect to MongoDB
    logger = MongoLogger()

    # Select AI provider
    if choice == "1":

        api_key = os.getenv("GEMINI_API_KEY")
```

```
    if not api_key:
        raise ValueError("Gemini API key not found in .env")

    provider = GeminiProvider(api_key)

elif choice == "2":

    provider = OllamaProvider("llama3.2")

else:

    raise ValueError("Invalid selection. Choose 1 or 2.")

# Create common client
client = LLMClient(provider, logger)

# Get user prompt
prompt = input("\nEnter your question: ")

# Generate response
answer = client.generate(prompt)

print("\n========== AI RESPONSE ==========")

print(answer)

print("\n========== COMPLETED ==========")
```

```
except Exception as error:

    print("\nApplication error:", error)

finally:

    if logger is not None:
        logger.close()
```

5. requirements.txt

```
pytest
google-genai
python-dotenv
pymongo
requests
```

6. .env

```
# Gemini API key is not required for Ollama execution.
# Ollama runs locally on the system.
```

7. test_mongo.py

```
# test_mongo.py

from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017/",
    serverSelectionTimeoutMS=5000
)

try:
```

```
    client.admin.command("ping")

    print("MongoDB connection successful!")

    db = client["llm_monitoring"]

    collection = db["call_logs"]

    print("Database:", db.name)

    print("Collection:", collection.name)

except Exception as e:

    print("Connection failed:", e)

finally:

    client.close()
```

EXECUTION

Run the following commands in the VS Code terminal.

Step 1: Install Python dependencies

python -m pip install -r requirements.txt

Step 2: Verify MongoDB service

Get-Service MongoDB

Step 3: Test MongoDB connectivity

python test_mongo.py

Step 4: Download the Ollama model

ollama pull llama3.2

Step 5: Run the application

python main.py

Select option 2 to use Ollama.

Enter a prompt when requested to generate an AI response.

SAMPLE OUTPUT

```
(.venv) PS C:\Users\sahas\OneDrive\Desktop\Btech\GenAI\Lab\Week3> python main.py
================================
       WEEK 3: LLM MONITORING
================================
1. Gemini
2. Ollama

Select provider (1/2): 2
Connected to MongoDB successfully!

Enter your question: What is the capital of japan?
Call details saved in MongoDB!

========== AI RESPONSE ==========
The capital of Japan is Tokyo.
```

MONGODB OUTPUT


```
_id: ObjectId('6af42277424f0c75f302f021')
provider: "OllamaProvider"
model: "llama3.2"
prompt: "What is the capital of japan?"
input_tokens: 32
output_tokens: 8
total_tokens: 40
latency_ms: 2685.11
cost: 0
attempts: 1
status: "success"
error: null
timestamp: ISODate('2026-11-11T08:08:23.804+00:00') 11 November 2026 at 08:08:23 UTC
```

```
_id: ObjectId('6af424487d6e85cc355222b5')
provider: "OllamaProvider"
model: "llama3.2"
prompt: "What is a Data Base"
input_tokens: 30
output_tokens: 521
total_tokens: 551
latency_ms: 20394.74
cost: 0
attempts: 1
status: "success"
error: null
timestamp: ISODate('2026-11-11T08:16:08.795+00:00') 11 November 2026 at 08:16:08 UTC
```


RESULT

Successfully developed and executed an LLM Monitoring System using Python, MongoDB, and Ollama. The application generates AI responses using the Llama 3.2 model and records important usage details, including token consumption, response latency, estimated cost, and request status in the MongoDB database.

MongoDB connectivity and successful LLM request logging were verified using MongoDB Compass.